

N Mixed-Effects Models

One way to combine all the statistical tests above is to fit a linear mixed-effects model where we treat the condition as the fixed effect and other factors including reviewer and idea as random effects, while also accounting for the differences among different topics. This way, we can rely on the regression to account for all the possible confounders as the random effects. Specifically, for each metric, we fit the following linear mixed-effects model:

```
model = smf.mixedlm("Score ~ Condition", df,
                    groups=df["Topic"],
                    re_formula="~Condition",
                    vc_formula={"ReviewerID": "0 + C(ReviewerID)",
                               "IdeaID": "0 + C(IdeaID)"})
```

This mixed-effects model analyzes the relationship between *Score* and *Condition*, while accounting for the hierarchical structure of the data. Fixed effects estimate the average effect of *Condition* on *Score*. Random intercepts for *Topic* allow for varying baseline scores across topics, and random slopes for *Condition* within each topic allow the effect of *Condition* to vary by topic. Additionally, variance components for *ReviewerID* and *IdeaID* account for variability in scores specific to individual reviewers and ideas, respectively.

The results are shown in Table 17. The intercepts in the mixed-effects models represent the estimated mean score of the baseline condition, which in this context is the *Human Ideas*. The coefficients for *Condition[AI Ideas]* and *Condition[AI Ideas + Human Rerank]* in the mixed-effects models represent the difference in the mean score for each metric between the AI ideas and the baseline (human ideas). For example, a positive coefficient of 0.761 for the novelty score means that *AI Ideas*, on average, score 0.761 points higher than *Human Ideas* on the novelty score metric; conversely, a negative coefficient of -0.330 for the feasibility score means that *AI Ideas*, score 0.330 points lower than *Human Ideas* on feasibility on average. The topic (group) variance in the mixed-effects model represents the variability in the outcome metric that can be attributed to differences between the topics, which is relatively small in general. Similarly, the idea variance and reviewer variance in the mixed-effects model represent the variability in the outcome metric that can be attributed to differences between individual ideas and between reviewers, respectively. The reviewer variances are high in general, suggesting that there is substantial variability in how different reviewers rate the same ideas. This implies that reviewer differences play a significant role in the observed scores, with some reviewers consistently giving higher or lower ratings.

Overall, the results from the mixed-effects models confirm our main conclusion that AI ideas are rated as significantly more novel than human ideas.

	Coef.	SE	<i>p</i>
Novelty Score			
Intercept	4.826	0.217	0.000***
Condition[AI Ideas]	0.756	0.331	0.023*
Condition[AI Ideas + Human Rerank]	0.902	0.305	0.003**
Idea Var	0.412	0.178	
Reviewer Var	0.803	0.202	
Excitement Score			
Intercept	4.493	0.212	0.000***
Condition[AI Ideas]	0.626	0.303	0.039*
Condition[AI Ideas + Human Rerank]	0.879	0.298	0.003**
Idea Var	0.495	0.227	
Reviewer Var	0.782	0.167	
Feasibility Score			
Intercept	6.595	0.224	0.000***
Condition[AI Ideas]	-0.300	0.294	0.307
Condition[AI Ideas + Human Rerank]	-0.183	0.314	0.561
Idea Var	0.476	0.188	
Reviewer Var	1.035	0.261	
Expected Effectiveness Score			
Intercept	5.156	0.211	0.000***
Condition[AI Ideas]	0.310	0.140	0.027*
Condition[AI Ideas + Human Rerank]	0.383	0.242	0.114
Idea Var	0.200	0.151	
Reviewer Var	0.469	0.141	
Overall Score			
Intercept	4.660	0.242	0.000***
Condition[AI Ideas]	0.137	0.294	0.640
Condition[AI Ideas + Human Rerank]	0.610	0.320	0.056
Idea Var	0.262	0.154	
Reviewer Var	1.071	0.225	

Table 17: Results of linear mixed-effects models. We **bold** results that are statistically significant (* $p < 0.05$; ** $p < 0.01$; *** $p < 0.001$). Our main conclusion on AI ideas being more novel than human ideas still holds here.

O Score Breakdown by Topic

We show the breakdown of all scores across all conditions by topic. Note that due to the smaller sample sizes for the per-topic breakdown, most results are not statistically significant and only offer an intuitive understanding of the trends.

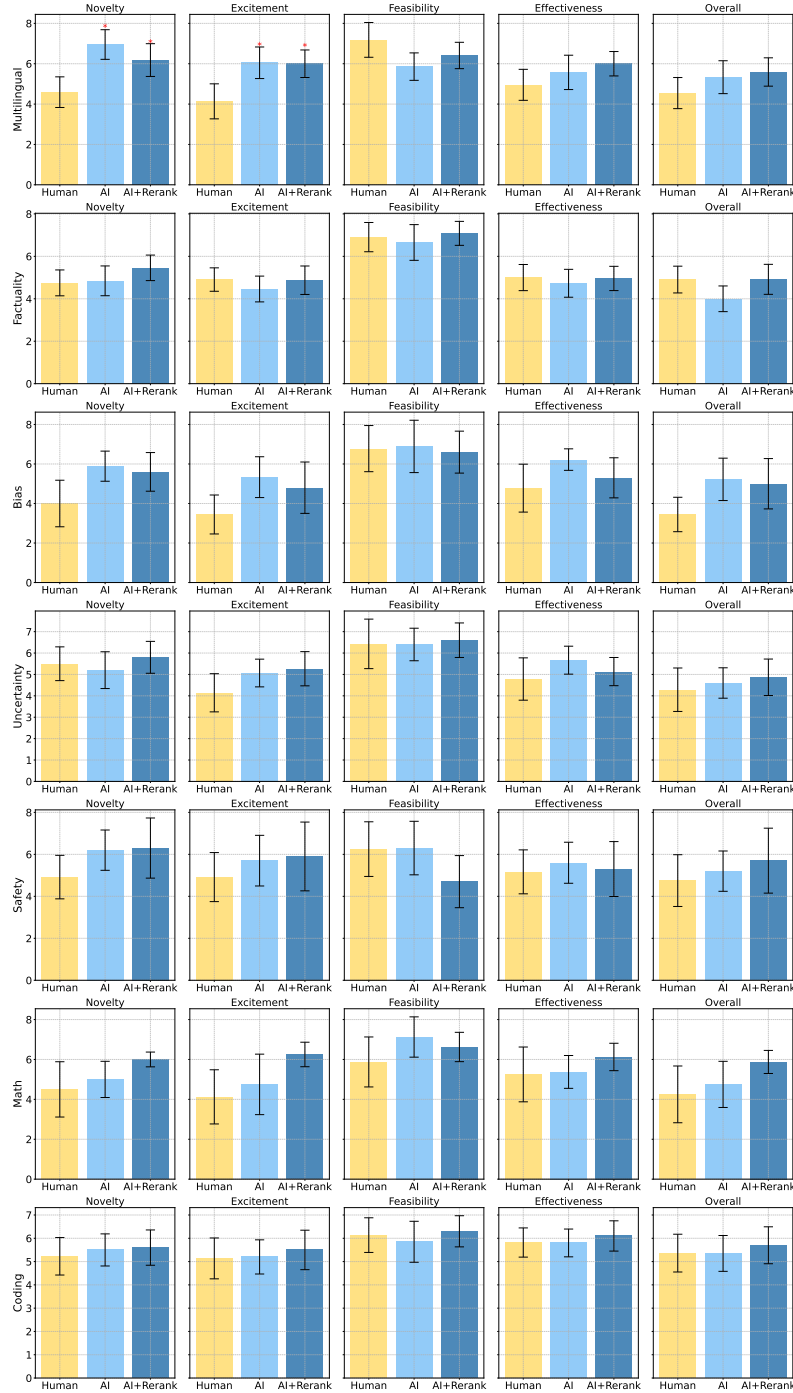


Figure 5: Breakdown of all scores by topic.

P Example Idea: Modular Calibration for Long-form Answers

Modular Calibration for Long-form Answers (Part 1)

1. Problem Statement: Calibrating the confidence of Large Language Models (LLMs) when generating long-form answers, such as essays and code, remains an open challenge in the field of natural language processing.

2. Motivation: While numerous methods have been developed to calibrate the performance of LLMs on multiple-choice questions or open-domain questions with short answers, extending these approaches to tasks requiring lengthy responses presents significant difficulties. For instance, in code generation tasks (e.g., the HumanEval dataset), traditional confidence extraction methods like perplexity may prove inadequate due to the substantial variation in answer length across questions. Verbalized confidence can be affected by instruction tuning artifacts or unclear scope, while the reliability of metrics such as Expected Calibration Error (ECE) and Macro-averaged Calibration Error (MacroCE) may be compromised by differences in task settings. Our aim is to propose a novel pipeline for confidence extraction and calibration of LLMs for long-form answers, drawing inspiration from methods used for short or fixed-set answers. This approach will enable us to monitor the model's long-form answer generation process and apply targeted external augmentation when necessary, thereby enhancing both performance and efficiency.

3. Proposed Method: We introduce Modular Calibration, a process comprising four core steps:

- 1. Extend:** Prompt the model to elaborate on the original question in relation to the answer, identifying which components of the question are addressed in the long-form response.
- 2. Decompose:** Instruct the LLM to break down the extended question and long-form answer into multiple modules.
- 3. Extract Confidence:** Utilize verbalized confidence or perplexity to determine the confidence level for each module.
- 4. Merge:** Based on the relationships between the modular questions/answers and the overall questions/answers, prompt the model to combine the modular confidence scores into an overall score representing the confidence in the long-form answer.

Each of these steps is executed by prompting the same LLM in different ways to elicit the desired response.

4. Step-by-Step Experiment Plan:

- 1. Gather Datasets:** Select datasets featuring long answers with correctness annotations. Potential candidates include GSM8K, Code Gen, and Essay Writing.
- 2. Construct Prompts:**
 - (a) Establish a baseline using direct prompting, where a query is presented without special techniques.
 - (b) Analyze outputs to refine prompts for the Extend and Decompose steps.
 - (c) For the Confidence step, employ vanilla perplexity or verbalized confidence extraction. If performance is unsatisfactory, explore advanced methods built upon these techniques, such as those presented in recent research (e.g., FaR paper).
- 3. Select Models:** Evaluate GPT-3.5 (Text-Davinci-003) and GPT-4 from the OpenAI API, as well as the open-source LLaMA-3-70B-chat.
- 4. Get Results:** Obtain confidence predictions from the models on the selected datasets using both baseline methods and the proposed Modular Calibration approach.
- 5. Analyze Results:** Compare the calibration performance of LLMs using the new method against the baselines (e.g., the perplexity of the entire long-form answer). Conduct qualitative and quantitative analyses on each component of the Modular Calibration process.

Modular Calibration for Long-form Answers (Part 2)

5. Test Case Examples:

- **Test Case 1: Verbalized Confidence Prompting**
 - Input: <Q> <A> Confidence (0-1)
 - Output: [Model generates a confidence score between 0 and 1]
- **Test Case 2: Modular Calibration Step 1 (Extend)**
 - Input: Given the answer, can you extend the question and elaborate on what points are covered in the answer?
 - Output: The answer covers these points of the question: (1) how fast A runs; (2) how fast B runs; (3) if A is faster than B.
- **Test Case 3: Modular Calibration Step 2 (Decompose)**
 - Input: Please decompose the above extended question and answers into modules.
 - Output:
 - * How fast A runs: [relevant excerpt from the original answer]
 - * How fast B runs: [relevant excerpt from the original answer]
 - [Additional modules as needed]
- **Test Case 4: Modular Calibration Step 3 (Extract)**
 - Input: How fast A runs: [relevant excerpt from the original answer] Confidence (0-1)
 - Output: 1. 0.9; 2. 0.6 [Additional confidence scores for other modules]
- **Test Case 5: Modular Calibration Step 4 (Merge)**
 - Input: For each of these points related to question X, the confidence is: 0.9, 0.6, ... What is the overall confidence for the whole problem?
 - Output: [Model generates an overall confidence score]

6. Fallback Plan: If the proposed Modular Calibration method does not demonstrate improvement over the baseline, we will execute each sub-question and module individually to assess whether calibration is enhanced for each component. This approach will facilitate debugging of the proposed method and potentially yield interesting insights into the relationships between performance/calibration of decomposed modules and overall problems. Alternatively, we may analyze the model's ability to effectively decompose questions and answers into appropriate modules. These analyses will inform potential refinements to the method or provide valuable insights into the limitations and capabilities of LLMs in handling complex, long-form responses.